EDA-AGENTS · DEEP DIVE

The Analog Loop

gm/ID + SPICE + autoresearch, end to end. Three agents, five topologies, one greedy loop.

PRESENTER

Mauricio Montanares

Scientist, IHP

DATE

2026·04·23

DECK

42 slides · 8 parts

SIBLING

digital.html

What you will learn

- The **gm/ID methodology** that underpins every analog decision eda-agents makes.
- The three analog agents in detail, plus the **CircuitTopology** contract that lets you add your own.
- The shared **autoresearch loop** — program.md, results.tsv, corner validation.
- Three CLI runtimes: Claude Code, **OpenCode**, and direct-Python.

PAIRED NOTEBOOKS

`demo/agents_miller_ota.ipynb`
~3 min, 6 SPICE evals, AutoresearchRunner only.

`demo/agents_analog_topology_to_sizing.ipynb`
~15 min; chains recommender → advisor → autoresearch → corners.

Eight parts, five-ish minutes each

01 The **analog design problem**

02 The **three analog agents**

03 CircuitTopology **architecture**

04 Topology **registry**

05 **AutoresearchRunner** — the loop

06 Skills + **MCP**

07 Three **CLI runtimes**

08 **Demo** walkthroughs + close

PART 01

The analog design problem

Tiny nets, PVT corners, topology choice. Why analog autoresearch looks nothing like hyperparameter search.

Why analog sizing is not a grid search

- **Tiny nets.** A typical block is 10–30 transistors — far fewer degrees of freedom than a digital flow, but each one matters.
- **Continuous parameters.** *W*, *L*, bias, mirror ratios. Snap-to-grid is still a continuum.
- **PVT corners.** Fast/slow process, ±10% voltage, temperature sweeps. Nominal is a sanity check, not a sign-off.
- **Topology choice is architectural.** Miller vs folded cascode vs telescopic is a design decision, not a knob.

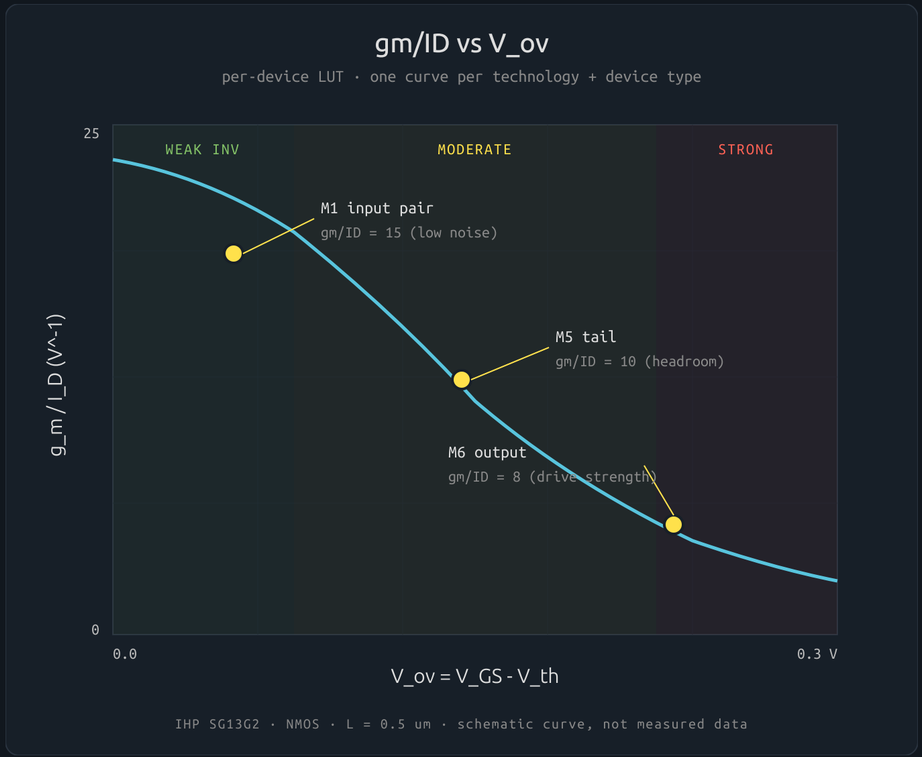
CONSEQUENCE

You cannot random-search this. Every evaluation must start from a **good** point. That is what gm/ID is for, and what the `analog-sizing-advisor` does with it.

gm/ID — the one-slide methodology

Precompute, per technology + per device, a lookup table of g_m/I_D (transconductance efficiency) vs $V_{GS} - V_{th}$ (overdrive).

- **High g_m/I_D** → weak inversion, large W, low noise, area cost.
- **Low g_m/I_D** → strong inversion, small W, fast, power cost.
- Pick per device; read back W/L and I_D from the table.
- Collapses the sizing problem to **one 1-D slider per device**.



What an analog agent loop needs

01 PROPOSE

Pick a topology + a sizing vector. Gets better with context.

02 SIMULATE

ngspice at nominal. Corners come later, post-convergence.

03 SCORE

Weighted FoM: gain, GBW, PM, power, area.

04 LOG

program.md + results.tsv. Human-readable and resumable.

05 ITERATE

Propose again conditioned on what we just learned.

PLUMBING, NOT MAGIC

Everything else in eda-agents on the analog side — skills, MCP tools, topology classes, LUT readers — is infrastructure for these five steps.

Cost asymmetry favours greedy refinement

PER SPICE EVALUATION

~ seconds

ngspice on 10–30 transistors. AC + DC analysis. Measurements parsed inline.

PER LLM PROPOSAL

~ cents

Gemini Flash or GLM-4.5-Flash at typical context. Full run budget well under $1.

CONSEQUENCE FOR SEARCH

You can afford 20–50 SPICE evaluations per run. You *cannot* afford random exploration. Every proposal must be **informed** by the current best and the learned insights from prior evaluations — which is exactly what greedy refinement from a gm/ID seed gives you.

PART 02

The three analog agents

Each a thin slice on top of the same CircuitTopology + SpiceRunner + AutoresearchRunner.

analog-topology-recommender

ANALOG AGENT · HOST-SIDE

- **Input:** one-line natural-language spec.
- **Output:** single JSON object — topology, rationale, starter_specs, confidence.
- **Refuses to invent.** If nothing matches, returns `custom` with confidence < 1, not a fabricated option.
- Registered under `.claude/agents/` and `.opencode/agent/` — same spec, two runtimes.

SKILLS IT LEANS ON

```
analog.idea_to_topology  # NL spec -> topology name
describe_topology          # MCP tool: registry lookup
```

TYPICAL INVOCATION

```
# Claude Code
/agents analog-topology-recommender

# OpenCode, headless
opencode run --agent analog-topology-recommender \
  "60 dB gain, 10 MHz GBW, rail-to-rail OTA"
```

Recommender output — JSON only

SPEC → OUTPUT

```
# Input
spec: "Rail-to-rail OTA, 10 MHz GBW, 60 dB gain, load 1 pF, IHP SG13G2"

# Output (the JSON the agent is contractually required to emit)
{
  "topology": "miller_ota",
  "rationale": "Miller compensation gives a predictable -20 dB/dec roll-off at the "
               "frequencies asked for. Folded cascode would give more headroom "
               "at the cost of an extra bias leg.",
  "starter_specs": {
    "gbw_hz": 10.0e6,
    "pm_deg":  60,
    "cl_f":    1.0e-12,
    "pdk":     "ihp_sg13g2"
  },
  "confidence": 0.82
}
```

analog-sizing-advisor

ANALOG AGENT · GM/ID-DRIVEN

- **Input:** chosen topology + target specs + gm/ID LUTs.
- **Output:** starting sizing vector that satisfies first-order constraints *before* any SPICE call.
- Hands off to `AutoresearchRunner` for greedy refinement.
- Turns blind random-search into well-conditioned local optimisation.

SKILLS IT LEANS ON

```
analog.gmid_sizing    # gm/ID cheat sheet
describe_topology      # design space / defaults
evaluate_topology      # one SPICE eval on demand
run_autoresearch       # hand off to the greedy loop
```

LUT SOURCES

`IHP SG13G2 · $EDA_AGENTS_IHP_LUT_DIR (external clone)`
`GF180MCU · auto-download, ~/.cache/eda-agents/gmid_luts/`

Advisor output — first-order sized, no SPICE yet

```yaml
# Starting sizing vector for a Miller OTA
M1_M2:  # input pair
  W: 18e-6  L: 0.5e-6
  gm_ID: 15   # low noise
M3_M4:  # active load
  W: 4e-6   L: 0.5e-6
M5:     # tail
  W: 8e-6   L: 1.0e-6
  gm_ID: 10   # headroom
M6:     # output stage
  W: 40e-6  L: 0.5e-6
  gm_ID: 8    # drive strength
ibias: 40e-6
Cc:    2.0e-12
```

WHAT JUST HAPPENED

- gm/ID per transistor was picked from the spec (noise → high, headroom → mid, drive → low).
- LUT returned W/L and I_D for each.
- Miller cap sized for target PM = 60° using the textbook formula.
- Ready to hand to the autoresearch runner — no wasted SPICE on random starting points.

FIRST SPICE CALL CONFIRMS

Measures Adc, GBW, PM, power at nominal. If these are within ~20% of the starter specs, greedy refinement takes over. If not, the advisor revises (usually a gm/ID +/- 2 step).

gf180-docker-analog

SIGNOFF AGENT · DOCKER-DRIVEN

- **Input:** frozen sizing + topology class + target PDK.
- **Output:** DRC-clean, LVS-clean layout on GF180MCU inside `hpretl/iic-osic-tools`.
- Uses `gLayout` for parametric placement, `Magic` for DRC, `Netgen` for LVS.
- Never touches container state directly — every action is a `docker exec` call.

SKILLS IT LEANS ON

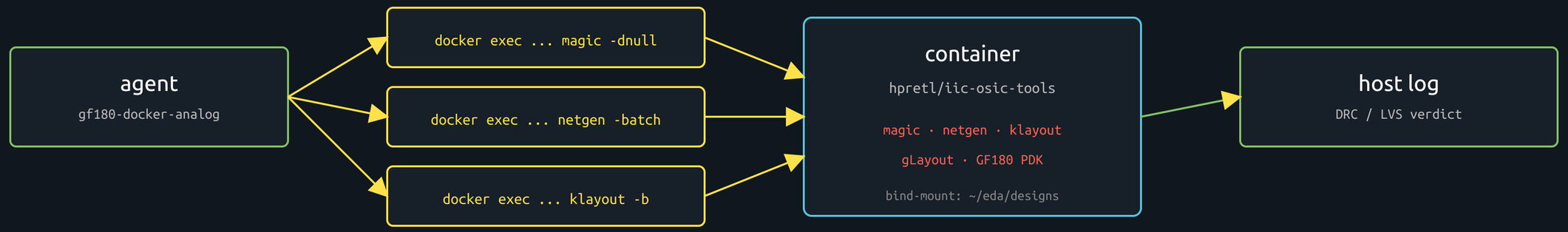
```
flow.analog_signoff_gf180_docker
flow.drc_checker
flow.drc_fixer
flow.lvs_checker
```

INVOCATION

```
# Claude Code
/agents gf180-docker-analog

# OpenCode, headless (e.g. CI)
opencode run --agent gf180-docker-analog \
  "Signoff the Miller OTA at ./layout/miller_ota.gds"
```

The docker-driver pattern



A DRC hit triggers `flow.drc_fixer` *(not a blanket retry). The fix lands in the same repository where you ran the agent — container is disposable, artifacts are yours.*

PART 03

CircuitTopology architecture

One abstract class, six methods, zero hardcoded topology names in the harness.

The `CircuitTopology` contract

```python
class CircuitTopology(ABC):
    # what the search space looks like
    def design_space(self) -> dict[str, Parameter]: ...
    def default_params(self) -> dict[str, Any]: ...

    # params -> concrete sizing -> ngspice deck
    def params_to_sizing(self, params) -> Sizing: ...
    def generate_netlist(self, sizing, work_dir) -> Path: ...

    # SPICE result -> score + validity
    def compute_fom(self, spice_result, sizing) -> float: ...
    def check_validity(self, spice_result) -> list[str]: ...

    # metadata the harness reads
    def topology_name(self) -> str: ...
    def specs_description(self) -> str: ...
```

Source: src/eda_agents/core/topology.py

The harness derives everything from metadata

- **Prompts:** built at runtime from `specs_description()` + `design_space()`. No hardcoded names.
- **Tool specs:** the MCP `describe_topology` / `evaluate_topology` tools dispatch on a class lookup.
- **Validity logic:** `check_validity()` is the single source of truth for "did this sizing meet spec".
- **FoM:** each topology owns its weighting — one class, one FoM, no global constants.

WHAT THIS BUYS YOU

Add a new topology tomorrow — every existing agent (recommender, sizing advisor, autoresearch) works against it with **zero** glue code. The harness is topology-agnostic by construction, not by promise.

Adding your own topology

FILE STRUCTURE

```
src/eda_agents/topologies/
  my_block.py                # ~200 lines total
    ├ netlist_template.cir   # ~100 lines
    ├ design_space(...)      # ~50 lines
    ├ compute_fom(...)       # ~30 lines
    └ check_validity(...)    # ~20 lines
```

RUNTIME REGISTRATION

```python
from eda_agents.topologies.my_block import MyTopology

topo = MyTopology()
runner = AutoresearchRunner(topology=topo, budget=20)
await runner.run(work_dir)
```

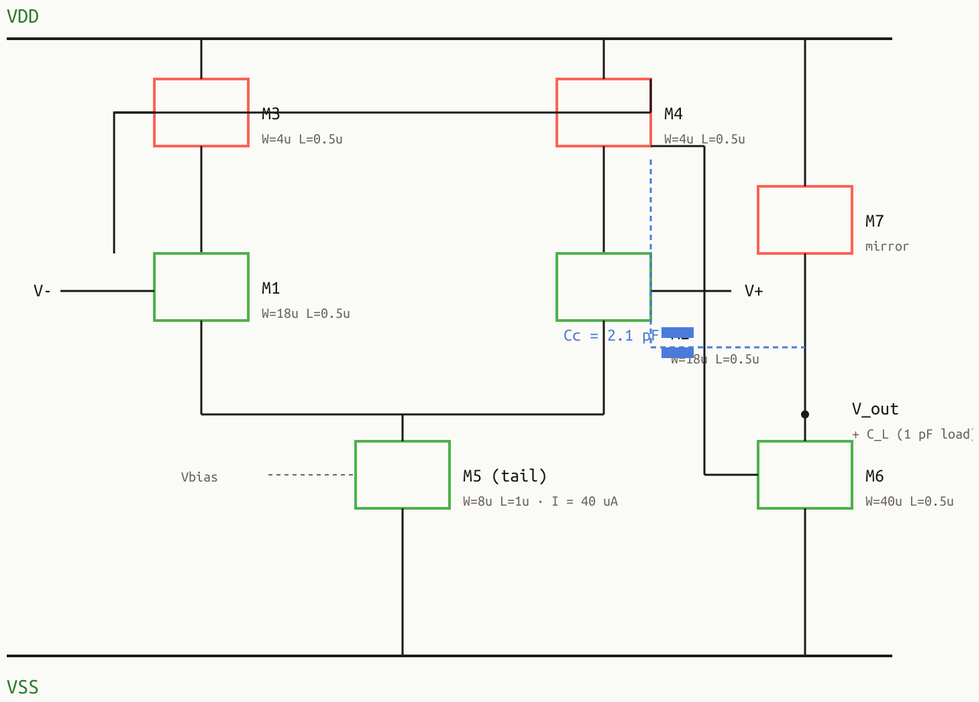
No registry JSON to edit. No rebuild. Import and go.

PART 04

Topology registry

Five topologies ship today. All five reachable from the same three agents.

Miller OTA — `ota_miller`



- **PDK:** IHP SG13G2 (130 nm, V_{DD} = 1.2 V).
- **Topology:** two-stage compensated OTA, NMOS input pair, Miller cap between stages.
- **Design space:** 5 transistor W/L + bias + Miller cap.
- **Target specs:** GBW 10 MHz, PM 60°, A_{dc} 60 dB, C_L 1 pF.

DESIGN_SPACE() · EXCERPT

```
{
  "M1_W": (1e-6, 50e-6, "log"),
  "M5_W": (1e-6, 20e-6),
  "ibias":(10e-6, 100e-6),
  "Cc":   (0.5e-12, 5e-12)
}
```

Source: `src/eda_agents/topologies/ota_miller.py`

AnalogAcademy OTA — `ota_analogacademy`

- **PDK:** IHP SG13G2.
- **Topology:** PMOS-input variant from the IHP AnalogAcademy course.
- Same shape as Miller OTA; different input polarity and bias scheme.
- Useful reference for porting a textbook example into the eda-agents registry.

WHY THIS SHIPS

The Academy course is the canonical analog-design teaching material for SG13G2. Having its OTA in the registry lets students reproduce the lab exercises with AI-driven sizing — same spec, same schematic, much less grid-search pain.

GF180 OTA — `ota_gf180`

- **PDK:** GF180MCU (180 nm, V_{DD} = 3.3 V).
- **Topology:** PMOS-input two-stage OTA targeting the GF180 BSIM4 models.
- Wafer-space PDK fork is *not* needed for a bare analog block.
- Design space shape mirrors the IHP OTAs — only the device models + voltage domain differ.

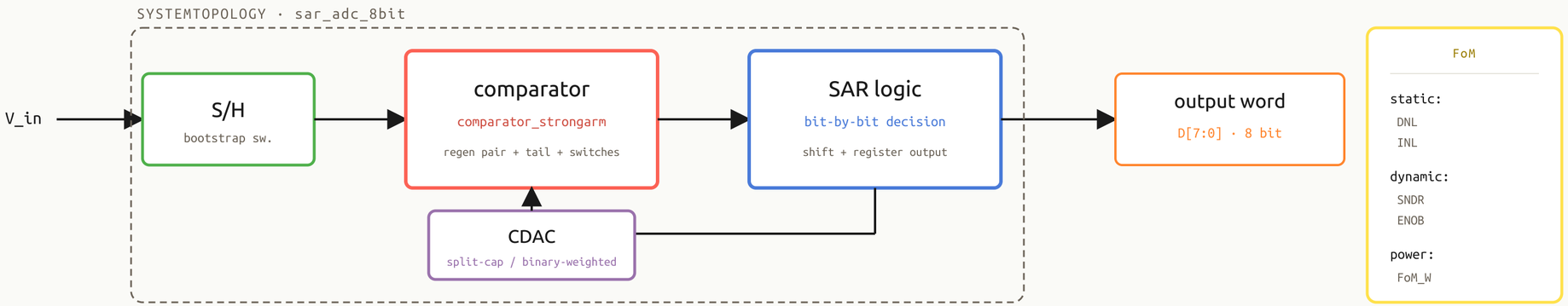
PDK-SPECIFIC PLUMBING

```
# netlist_lib_lines() returns the right lib directives
.lib '$PDK_ROOT/gf180mcuD/libs.tech/ngspice/
     sm141064_mim.ngspice' mim

# BSIM4 via built-in ngspice, no OSDI required
.model nfet_03v3 nmos (level=14 ...)
```

Source: `src/eda_agents/topologies/ota_gf180.py` and `src/eda_agents/core/pdk.py`

Dynamic comparator + SAR ADC



COMPARATOR_STRONGARM

Standalone CircuitTopology. Regen pair + tail + switches. FoM: offset, decision time, kickback. IHP + GF180.

SAR_ADC_8BIT · SYSTEMTOPOLOGY

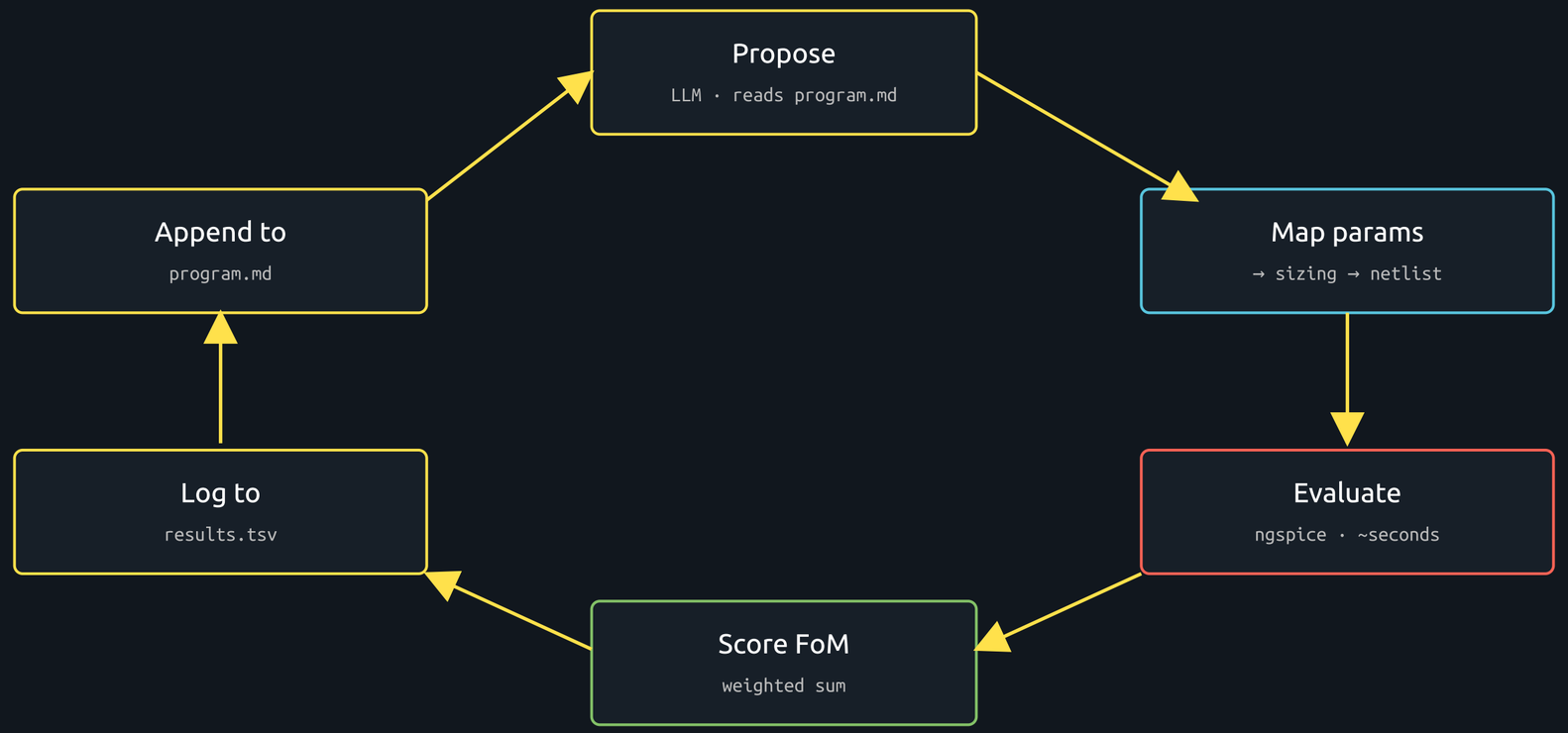
Composes comparator + CDAC + logic. One FoM over static (DNL/INL) + dynamic (SNDR/ENOB). Transistor & behavioural variants.

PART 05

AutoresearchRunner — the loop

Six nodes. Two persistence files. Tens of evaluations. Cents of LLM cost.

Six nodes, one cycle



program.md + results.tsv

NARRATIVE BRAIN · PROGRAM.MD

```
# Goal
Miller OTA on IHP SG13G2,
GBW ≥ 10 MHz, PM ≥ 60°.

# FoM
0.5·GBW + 0.3·PM − 0.2·power

# Current best (eval 07)
params: {M1.W: 18µ, ibias: 42µA, Cc: 2.1pF}
FoM: 0.71

# Learned insights
- Cc < 2 pF ruins PM.
- Tail > 50 µA loses gain faster than GBW improves.
```

NUMERIC HISTORY · RESULTS.TSV

eval	GBW_MHz	PM_deg	P_mW	FoM	status
01	6.2	72.1	0.18	0.41	valid
02	8.7	68.4	0.22	0.55	valid
03	11.4	55.3	0.29	0.47	PM fail
04	10.1	63.8	0.24	0.62	valid
05	12.6	61.2	0.31	0.68	valid
06	10.8	65.4	0.26	0.69	valid
07	11.2	64.1	0.27	0.71	best

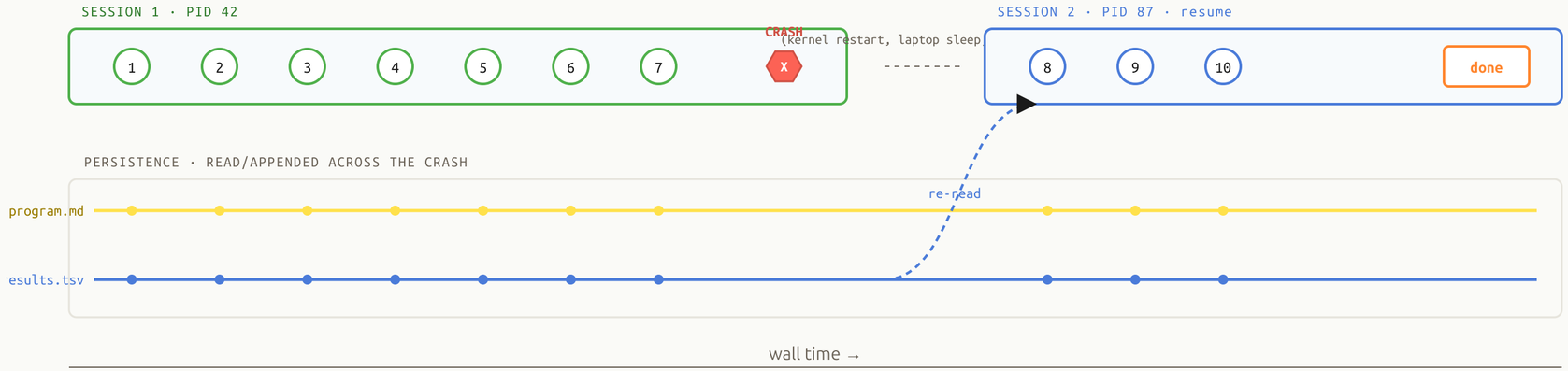
These are the only persistence files. No pickle. No hidden state. Delete them and the run restarts.

Budget sizing — how many evaluations?

SCENARIO	BUDGET	WALL TIME	LLM COST	USE FOR
rehearse	0	< 5 s	$0.00	dry-run — verify PDK + env
demo	6	~ 3 min	< $0.05	Miller OTA teaching notebook
default	20	~ 10 min	~ $0.15	standard refinement run
hunt	50	~ 25 min	~ $0.40	tough spec, need exploration
corners	+ 16	+ 8 min	$0.00	PVT sweep post-convergence

Wall time dominated by the inner SPICE loop — tens of transistors, seconds per run. LLM cost is trivial at these scales; the budget knob is about search breadth, not dollars.

Resumable runs — crash, kill, rebase



No pickle, no hidden state. Two files on disk. The repo tree is the whole state.

`analog.corner_validator` — after the loop converges

DEFAULT CORNER SWEEP

AXIS	POINTS	NOTES
Process	`tt, ff, ss, fs, sf`	five SPICE decks per candidate
Voltage	`−10%, nom, +10%`	three V_{DD} points
Temperature	`−40°C, 27°C, 85°C`	industrial range by default
Load	$C_L/2$, C_L, $2 \cdot C_L$	verifies PM margin under real loads

ON FAILURE

If the winner fails any corner, the validator does *not* accept it. The runner backs off to the previous candidate (row $n - 1$ in `results.tsv`) and re-validates. You never ship a nominal-only-passing design.

PART 06

Skills over MCP

Ten analog skills. Runtime discovery. One stdio server. Two clients.

The `analog.*` namespace

WORKHORSE

- `analog.gmid_sizing`
- `analog.explorer`
- `analog.corner_validator`
- `analog.orchestrator`
- `analog.idea_to_topology`

TOPOLOGY-SPECIFIC

- `analog.miller_ota_design`
- `analog.sar_adc_design`
- `analog.adc_metrics`
- `analog.behavioral_primitives`
- `analog.custom_composition`

Source: `src/eda_agents/skills/analog.py`

Runtime discovery — agents find skills, not the other way around

MCP TOOLS FOR DISCOVERY

```
# Enumerate everything the server exposes
list_skills() -> ["analog.gmid_sizing", "analog.explorer", ...]

# Fetch the rendered prompt for one
render_skill("analog.gmid_sizing") -> "# Skill: gm/ID..."

# Topology-aware variants
describe_topology("miller_ota") -> {design_space, specs, ...}
```

WHY THIS SHAPE

Agents are not built against a static skill list. They discover at runtime. Adding a new skill (or retiring one) does not require editing any agent's prompt.

MCP registration — one server, two clients

CLAUDE CODE · ~/.CONFIG/CLAUDE-CODE/MCP.JSON

```
{
  "mcpServers": {
    "eda-agents": {
      "type": "stdio",
      "command": "eda-mcp"
    }
  }
}
```

OPENCODE · ./OPENCODE.JSON AT REPO ROOT

```
{
  "$schema": "https://opencode.ai/config.json",
  "mcp": {
    "eda-agents": {
      "type": "local",
      "command": [".venv/bin/python",
                  "-m", "eda_agents.mcp"]
    }
  }
}
```

BOOTSTRAP

`eda-init` writes both files. Run it once after you clone the repo and activate the venv. Same `eda-mcp` stdio server serves both clients — write a skill once, use it everywhere.

PART 07

Three CLI runtimes

Pick by subscription. Same agents, same skills, same results under the hood.

Claude Code — `claude --print`

`claude`

Anthropic subscription

`interactive`

/agents slash command in the Claude Code session

`headless`

claude --print --output-format json, via ClaudeCodeHarness in Python

INTERACTIVE CALL

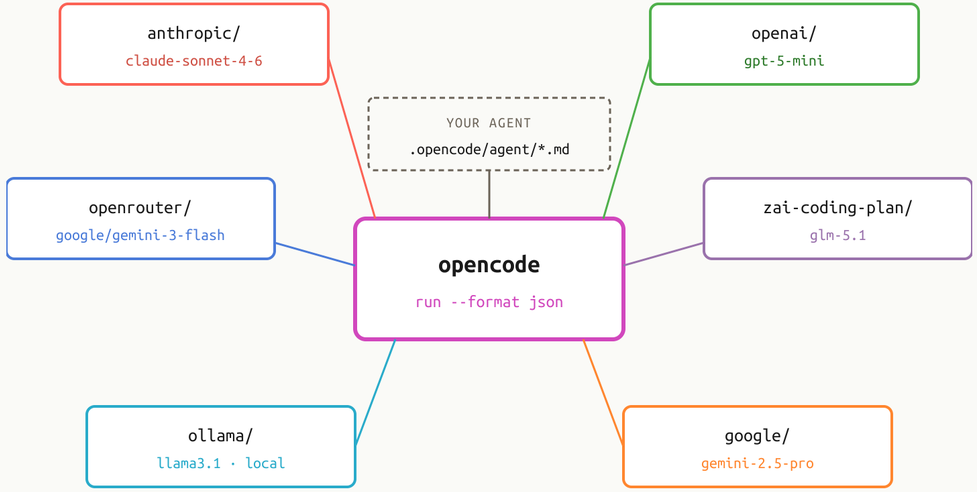
```
/agents analog-sizing-advisor
# pastes a focused prompt into your session, loads MCP skills,
# and gives you the tool rails for this agent.
```

OpenCode — `opencode run`

opencode
multi-provider: Anthropic, OpenAI, Gemini, OpenRouter, Ollama, Z.AI, Cursor, Windsurf

interactive
opencode --agent <name> — TUI with the same 5-agent roster under .opencode/agent/

headless
opencode run --format json, via OpenCodeHarness in Python



`-m <provider>/<model>`

one CLI · five+ providers · same agent roster

INSTALL ONCE

```
npm install -g opencode-ai
eda-init                          # writes opencode.json
```

PICK ANY PROVIDER AT CALL TIME

```
# Gemini Flash via OpenRouter (cheap)
opencode run --agent analog-sizing-advisor \
  -m openrouter/google/gemini-3-flash-preview \
  "Size a Miller OTA for 10 MHz GBW."

# Z.AI coding plan (GLM-5.1)
opencode run --agent analog-sizing-advisor \
  -m zai-coding-plan/glm-5.1 "..."
```

Direct Python — `AutoresearchRunner`

python
no CLI, no TUI — import, construct, await

deterministic
CI-friendly, no user in the loop, same skills + FoM

model choice
LiteLLM string (openrouter/..., zai/..., anthropic/...)

REFERENCE · EXAMPLES/07_AUTORESEARCH_CIRCUIT.PY

```python
from eda_agents.topologies.ota_miller import MillerOTATopology
from eda_agents.agents.autoresearch_runner import AutoresearchRunner

topo = MillerOTATopology()
runner = AutoresearchRunner(
    topology=topo,
    model="zai/GLM-4.5-Flash",
    budget=20,
)
result = await runner.run(Path("autoresearch_miller"))
print(result.summary)
```

PART 08

Demos · run it yourself

Two notebooks. Safe defaults. Same repo.

Demo 1 — `agents_miller_ota.ipynb`

FIVE STEPS

1. Editable install — `pip install -e .`
2. Environment check — ngspice, PDK_ROOT, LUTs, API key
3. Instantiate `MillerOTATopology`
4. One SPICE eval at default params (sanity)
5. `AutoresearchRunner(budget=6)` — 2–3 min

SAFE BY DEFAULT

Every `RUN_*` flag defaults to `False`. Opening the notebook is a no-op — it prints what it *would* do. Flip `RUN_AUTORESEARCH=True` when your environment check came out clean.

Demo 2 — `agents_analog_topology_to_sizing.ipynb`

THE FULL CHAIN

1. Editable install + env check.
2. Send a natural-language spec to `analog-topology-recommender`.
3. Feed its JSON into `analog-sizing-advisor`.
4. Hand the starter sizing to `AutoresearchRunner`.
5. Pass the winner through `analog.corner_validator`.
6. Tail `program.md` and inspect the PVT verdict.

WHAT YOU SEE AT THE END

```
# Chained result
recommender  → miller_ota @ conf=0.82
advisor       → W=18µ, L=0.5µ, ibias=40µA
autoresearch  → best FoM 0.71 @ eval 7/12
corners       → pass at 12/15 points
               → back off to eval 6, retry
final         → accepted eval 6, FoM 0.69
```

The analog loop is infrastructure.

gm/ID is the methodology. CircuitTopology is the contract. AutoresearchRunner is the loop.

`github.com/Mauricio-xx/eda-agents` · *sibling deck:* `digital.html`